Movie Genre Classification

Problem Statement

Background Information: In the age of digital media and streaming platforms, the movie industry generates a vast amount of data, including movie scripts, reviews, and user ratings. Analyzing this data can provide valuable insights into audience preferences, movie genres, and content recommendation systems. Movie genre classification is a task that aims to automatically assign genres to movies based on their textual content, such as scripts or plot summaries. This task has practical applications in various areas, including movie recommendation systems, content tagging, and genre-based content analysis.

Dataset Information: The dataset used in this project is a collection of movie scripts and their corresponding genres. It contains information about the movie text and the assigned genre. The dataset is in CSV format and includes the following columns:

- id: A unique identifier for each movie script.
- text: The textual content of the movie script.
- genre: The assigned genre of the movie.

The dataset consists of a diverse range of movie genres, including action, adventure, comedy, drama, horror, romance, sci-fi, thriller, and others. The goal of this project is to build a machine learning model that can accurately classify movie scripts into their respective genres based on the provided textual content.

Project Objective: The objective of this project is to develop a movie genre classification system that can automatically assign genres to movie scripts. By leveraging natural language processing techniques and machine learning algorithms, the model will learn to identify patterns and features in the text data that distinguish different

genres. The accuracy of the model will be evaluated using appropriate evaluation metrics, such as accuracy score and confusion matrix.

Expected Deliverables: The expected deliverables of this project include:

- Data loading and exploratory data analysis (EDA): Load the movie script dataset, examine its structure, and gain insights into the distribution of genres.
- Data cleaning and preprocessing: Clean and preprocess the textual data by removing unnecessary characters, converting to lowercase, removing stopwords, and performing stemming.
- Feature extraction: Convert the preprocessed text data into numerical feature vectors using techniques like Bag-of-Words or TF-IDF.
- Model selection and training: Choose a suitable machine learning algorithm (e.g., Naive Bayes, SVM, or deep learning models) and train it on the extracted features using a training dataset.
- Model evaluation: Evaluate the trained model's performance using appropriate evaluation metrics and analyze the results to assess the accuracy and effectiveness of the classification system.
- Prediction and interpretation: Apply the trained model to predict genres for unseen movie scripts and interpret the results to gain insights into the model's performance and genre classification accuracy.

Framework

Loading Data:

- Import the necessary libraries, such as NumPy and pandas.
- Load the movie script dataset using the pd.read_csv() function and store it in a DataFrame called df.

EDA (Exploratory Data Analysis):

- Explore the dataset to gain insights into its structure and contents.
- Print the column names using df.columns.
- Determine the number of rows and columns in the dataset using df.shape.
- Display the first few rows of the dataset using df.head(10).
- Visualize the distribution of movie genres using a countplot with sns.countplot().

Data Cleaning and Preprocessing:

- Identify unique movie genres in the dataset and map them to numerical values using a genre mapper dictionary.
- Check for any NaN values in the dataset using df.isna().any().
- Remove the 'id' column from the dataset using df.drop('id', axis=1, inplace=True).
- Import the required libraries for natural language processing (NLP), such as NLTK.
- Clean and preprocess the movie script text data using the following steps:
- Iterate over each movie script in the dataset.
- Remove special characters from the script using regular expressions.
- Convert the script to lowercase.
- Tokenize the script into individual words.
- Remove stop words (commonly occurring words) from the script.
- Perform stemming (reducing words to their root form) on the remaining words.
- Join the processed words back into a single string, representing the cleaned script.
- Append the cleaned script to a corpus list.
- Print a subset of the corpus to verify the cleaning and preprocessing steps.

Data Visualization:

- Create separate lists for the cleaned scripts of specific genres (e.g., drama, action, comedy) by filtering the DataFrame based on genre labels.
- Concatenate the cleaned scripts of each genre to form a single string for word cloud generation.
- Generate word clouds using the WordCloud library to visualize the most frequent words associated with each genre.

Feature Extraction:

- Import the CountVectorizer class from the sklearn.feature_extraction.text module.
- Initialize a CountVectorizer object (cv) with parameters like maximum number of features and n-gram range.
- Apply the CountVectorizer's fit_transform() method on the corpus to obtain the feature vectors (X) representing the text data.
- Assign the genre labels to the target variable (y) by extracting the 'genre' column from the DataFrame.

Model Building:

- Import necessary libraries for model building, such as train_test_split from sklearn.model_selection.
- Split the dataset into training and testing sets using train_test_split().
- Initialize a Multinomial Naive Bayes classifier object (nb_classifier) from the sklearn.naive_bayes module.
- Fit the classifier to the training data using nb_classifier.fit().
- Make predictions on the test set using nb_classifier.predict().
- Evaluate the accuracy of the model using metrics like accuracy_score and confusion_matrix.
- Perform hyperparameter tuning on the classifier by varying the alpha parameter to improve performance.

Predictions:

- Define a function (genre_prediction) that takes a sample movie script as input and predicts its genre.

- Preprocess the sample script using the same cleaning and preprocessing steps as before.
- Transform the preprocessed script into a feature vector using cv.transform().
- Use the trained classifier to predict the genre of the sample script using nb_classifier.predict().
- Display the predicted genre to the user.

Conclusion and Future Improvements:

- Summarize the findings and conclusions from the project.
- Discuss potential future improvements, such as trying different classification algorithms, feature engineering techniques, or incorporating additional data sources.

Code Explanation

The code you provided for the Movie Genre Classification project consists of several functions and steps that work together to accomplish the task. Here's a breakdown of each function and the workflow of the code:

clean_text(text) function:

- This function takes a text parameter, which represents a movie script, and performs cleaning and preprocessing operations on it.
- It removes special characters, converts the text to lowercase, tokenizes it into individual words, removes stop words, performs stemming on the words, and joins them back into a single string.
- The function returns the cleaned and preprocessed text.

main() function:

- This is the main function that orchestrates the entire workflow of the code.
- It starts by loading the movie script dataset from a CSV file using pandas and storing it in a DataFrame called df.
- Next, it performs some exploratory data analysis (EDA) to understand the dataset's structure and contents. This includes printing column names, checking the number of rows and columns, and displaying the first few rows of the dataset.
- The function then defines a genre mapper dictionary to map movie genres to numerical values.
- It checks for any missing values in the dataset and drops the 'id' column, which is not required for classification.
- After that, the function imports the necessary libraries for natural language processing (NLP), such as NLTK.
- It iterates over each movie script in the dataset and applies the clean_text() function to clean and preprocess the text.
- The cleaned scripts are appended to a corpus list.
- The function then generates word clouds to visualize the most frequent words associated with each genre.
- Next, it performs feature extraction using the CountVectorizer class from the sklearn library. This converts the text data into numerical feature vectors.

- The dataset is split into training and testing sets using train_test_split from sklearn.
- A Multinomial Naive Bayes classifier is initialized, trained on the training data, and used to make predictions on the test data.
- The accuracy of the model is evaluated using metrics such as accuracy_score and confusion_matrix.
- Finally, the function defines a genre_prediction() function that takes a sample movie script as input, preprocesses it, and uses the trained classifier to predict its genre.

Workflow of the code:

- The code starts by loading the dataset and performing EDA to gain insights into the data.
- It then cleans and preprocesses the movie script text using the clean_text() function, building a corpus of cleaned scripts.
- Word clouds are generated to visualize the most frequent words associated with each genre.
- Feature extraction is performed using CountVectorizer to convert the text data into numerical feature vectors.
- The dataset is split into training and testing sets.
- A Multinomial Naive Bayes classifier is trained on the training data and used to make predictions on the test data.
- The accuracy of the model is evaluated.
- Finally, the user can provide a sample movie script, and the code will predict its genre using the trained classifier.

Future Work

1. Data Augmentation:

- One potential improvement is to augment the dataset by generating synthetic data. This can be done by applying techniques such as text synthesis, back-translation, or word replacement to create additional movie scripts.
- To implement this, you can explore libraries like NLTK or spaCy for text synthesis or consider using pre-trained language models like GPT-3.5 to generate synthetic movie scripts.

2. Advanced Text Preprocessing:

- Enhance the text preprocessing step to further improve the quality of the cleaned scripts.
- Explore additional techniques such as lemmatization, spell checking, or advanced tokenization methods like word embeddings (e.g., Word2Vec or GloVe) to capture semantic relationships between words.
- Implement these techniques using libraries like NLTK, spaCy, or gensim.

3. Advanced Feature Extraction:

- Explore advanced feature extraction techniques to capture more meaningful information from the movie scripts.
- Consider using TF-IDF (Term Frequency-Inverse Document Frequency) or word embeddings (e.g., Word2Vec or GloVe) to represent the textual features.
- Implement these techniques using libraries like scikit-learn or gensim.

4. Experiment with Different Classifiers:

- Try using different classification algorithms apart from Naive Bayes, such as Support Vector Machines (SVM), Random Forest, or Gradient Boosting algorithms (e.g., XGBoost or LightGBM).
- Experiment with different hyperparameter settings to find the best-performing classifier for the movie genre classification task.
- Implement these algorithms using scikit-learn or other machine learning libraries.

5. Model Evaluation and Performance Metrics:

- Explore additional evaluation metrics to assess the performance of the classification model.
- Consider metrics like precision, recall, F1-score, or area under the receiver operating characteristic curve (ROC AUC) to gain deeper insights into the model's performance.
- Implement these metrics using scikit-learn or other evaluation libraries.

6. Cross-Validation and Hyperparameter Tuning:

- Implement cross-validation techniques, such as k-fold cross-validation, to obtain a more robust estimate of the model's performance and generalize better to unseen data.
- Perform hyperparameter tuning using techniques like grid search or random search to find the optimal hyperparameter values for the classification model.
- Implement these techniques using scikit-learn or other model selection libraries.

7. Model Deployment and API Integration:

- Deploy the trained classification model as an API to allow users to classify movie genres programmatically.
- Use frameworks like Flask or Django to develop a RESTful API that can receive movie scripts as input and return predicted genres.
- Host the API on cloud platforms like AWS, Google Cloud, or Heroku for easy accessibility.

By following this step-by-step guide and implementing the suggested future work, you can enhance the Movie Genre Classification project by incorporating advanced techniques, improving the model's performance, and deploying it as a functional API. Remember to document each step and experiment with different approaches to find the best solutions for your specific use case.

Exercise

1. Question: What are some potential challenges or limitations of using Naive Bayes for movie genre classification? How can these challenges be addressed?

Answer: Naive Bayes is a simple and effective algorithm for text classification, but it has certain limitations:

- **Challenge 1:** Independence assumption: Naive Bayes assumes that the features (words) are independent of each other, which may not hold true for movie scripts where word dependencies exist. This assumption can affect the model's performance.
- **Challenge 2:** Handling unseen words: Naive Bayes assigns zero probability to words that it hasn't seen in the training data, leading to issues with unseen words during classification.

To address these challenges:

- **Solution 1:** Feature engineering: Instead of using individual words as features, we can use more meaningful features like n-grams (sequences of words) or word embeddings to capture word dependencies and semantic relationships.
- **Solution 2:** Smoothing techniques: Implement smoothing techniques like Laplace smoothing (additive smoothing) to assign non-zero probabilities to unseen words, preventing zero probabilities and allowing the model to make predictions for unseen words.

2. Question: What are some alternative evaluation metrics to accuracy that can be used to assess the performance of the movie genre classification model? When would you prefer to use these metrics?

Answer: Accuracy is a commonly used metric, but there are other evaluation metrics that provide additional insights. Some alternative metrics include:

- Precision: Precision is the ratio of correctly predicted positive instances (true positives) to the total instances predicted as positive. It measures the model's ability to correctly identify positive cases.

- Recall: Recall, also known as sensitivity or true positive rate, is the ratio of correctly predicted positive instances (true positives) to the total actual positive instances. It measures the model's ability to identify all positive cases.
- F1-score: F1-score is the harmonic mean of precision and recall. It provides a balanced measure between precision and recall.
- ROC AUC: ROC AUC (Receiver Operating Characteristic Area Under the Curve) measures the trade-off between true positive rate and false positive rate. It provides a measure of the model's ability to distinguish between classes.

These metrics are useful in different scenarios. For example:

- If false positives are more critical (e.g., misclassifying a drama movie as an action movie), precision is a suitable metric to focus on.
- If false negatives are more critical (e.g., misclassifying an action movie as a drama movie), recall is important.
- F1-score provides a balanced measure when precision and recall are both important.
- ROC AUC is useful when there is a class imbalance or when the cost of false positives and false negatives is similar.

3. Question: How can you improve the performance of the movie genre classification model without changing the underlying algorithm?

Answer: There are several techniques to improve the performance of the model without changing the algorithm:

- **Data preprocessing:** Enhance the text preprocessing step by incorporating techniques like lemmatization, spell checking, or advanced tokenization methods to improve the quality of the cleaned scripts.
- **Feature engineering:** Experiment with different feature extraction techniques such as TF-IDF or word embeddings to capture more meaningful information from the movie scripts.
- **Hyperparameter tuning:** Optimize the hyperparameters of the model using techniques like grid search or random search to find the best combination for improved performance.

- **Ensemble learning:** Combine predictions from multiple models (e.g., Naive Bayes, SVM, or Random Forest) to leverage their individual strengths and improve overall performance.
- **Error analysis:** Analyze misclassified instances and identify common patterns or sources of errors. Use this information to refine the preprocessing, feature extraction, or model selection process.

4. Question: How would you handle class imbalance in the movie genre classification dataset? What techniques can be employed to address this issue?

Answer: Class imbalance occurs when the number of instances in different classes is significantly different. In movie genre classification, it's common to have an uneven distribution of genres. To address class imbalance:

- Data resampling: Employ techniques like oversampling the minority class (e.g., Synthetic Minority Over-sampling Technique or SMOTE) or undersampling the majority class to balance the class distribution.
- Class weights: Assign higher weights to the minority class during model training to emphasize its importance and reduce the bias towards the majority class.
- Generate synthetic data: Generate synthetic data for the minority class using techniques like text synthesis or data augmentation to increase the representation of the underrepresented class.
- Ensemble methods: Use ensemble methods like bagging or boosting that can handle class imbalance naturally by creating diverse subsets or assigning higher weights to misclassified instances.

5. Question: How would you deploy the movie genre classification model as an API for real-time predictions?

Answer: Deploying the movie genre classification model as an API allows users to make predictions programmatically. Here's a step-by-step guide:

- **Step 1:** Model serialization: Serialize the trained model and save it to a file for easy retrieval.
- **Step 2:** API development: Use a web framework like Flask or Django to develop a RESTful API that can receive HTTP requests.
- **Step 3:** Input validation: Implement input validation to ensure that the incoming movie script data is in the expected format and within the allowed size limits.

- **Step 4:** Model loading: Load the serialized model into memory when the API starts.
- **Step 5:** Prediction endpoint: Create an endpoint that receives movie scripts as input, preprocesses the text, and makes predictions using the loaded model.
- **Step 6:** Output formatting: Format the prediction results in a suitable response format (e.g., JSON) and send it back to the user.
- **Step 7:** API deployment: Deploy the API on a cloud platform like AWS, Google Cloud, or Heroku for easy accessibility and scalability.

By following these steps, the movie genre classification model can be deployed as an API, enabling users to interact with it and obtain real-time predictions for movie scripts.